Detalles de la tarea de la unidad.

Enunciado.

El ejercicio va a ir sobre la base de datos EquinoTEIS y con los datos instalados vamos a realizar pruebas asociadas a los seis fenómenos asociados a la Concurrencia (dirty read, dirty write, non-repeatable read, lost update, phantom read, write skew), cuyos código de ejemplo para trabajar los tienes ya incluidos en el fichero "EquinoTEIS – 5. Ejercicio de Concurrencia.sql".

Los pasos que deberás seguir los tienes también ajuntos, pero son los siguientes:

-- 1. Abrir dos sesiones simultáneas: T1 y T2

-- * En Oracle -> emplea sesiones SQL plus

-- * En MySQL -> emplea sesiones MySQL 8.0 Command Line Client

-- 2. Inicia en el nivel de aislamiento que desees en ambas sesiones T1 y T2, esperando el Query Ok. Ten en cuenta que sea válida en Oracle o en MySQL.

-- 3. Ejecuta ahora los comandos asociados (u otros similares) a cada Fenómeno paso a paso. Ten cuidado al ejecutarlos para segurar que sea en la sesión indicada.

-- 4. Realiza las capturas de pantalla que consideres adecuadas.

-- 5. En resultado indica si es el esperado al Fenómeno asociado, y si no lo es indica a qué se debe y cómo se podría evitar.

-- 6. Si quieres, puedes comparar resultados en Oracle vs MySQL.

Criterios de puntuación. Total 1 punto.

- **Ejercicio:** 1,00 puntos
  - Funcionamiento correcto de cada Fenómeno y justificación en el campo Resultado asociada, valorado en 1/6 de punto (0,1666).
  - Se valuará la correcta sintaxis, el uso de la forma de inserción asignada, la consistencia con los datos proporcionados.

TOTAL: 1 punto

Recursos necesarios para realizar la tarea.

- Ordenador personal con, al menos 1 Gigabyte de memoria RAM.
- Oracle Database 18c XE instalado (o versión posterior).
- SQLDeveloper opcional, MySQL Developer.

Indicaciones de entrega.

- Una vez realizada la tarea, elaborarás un único documento donde figuren las respuestas correspondientes. El envío se realizará a través de la plataforma de la forma establecida para ello, y el archivo se nombrará siguiendo las siguientes pautas:

  apellido1_apellido2_nombre_SIGxx_Tarea
- Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así, por ejemplo, la alumna **Fulgencia Berdeal Outes** para la quinta unidad del MP de BD debería nombrar esta tarea como:

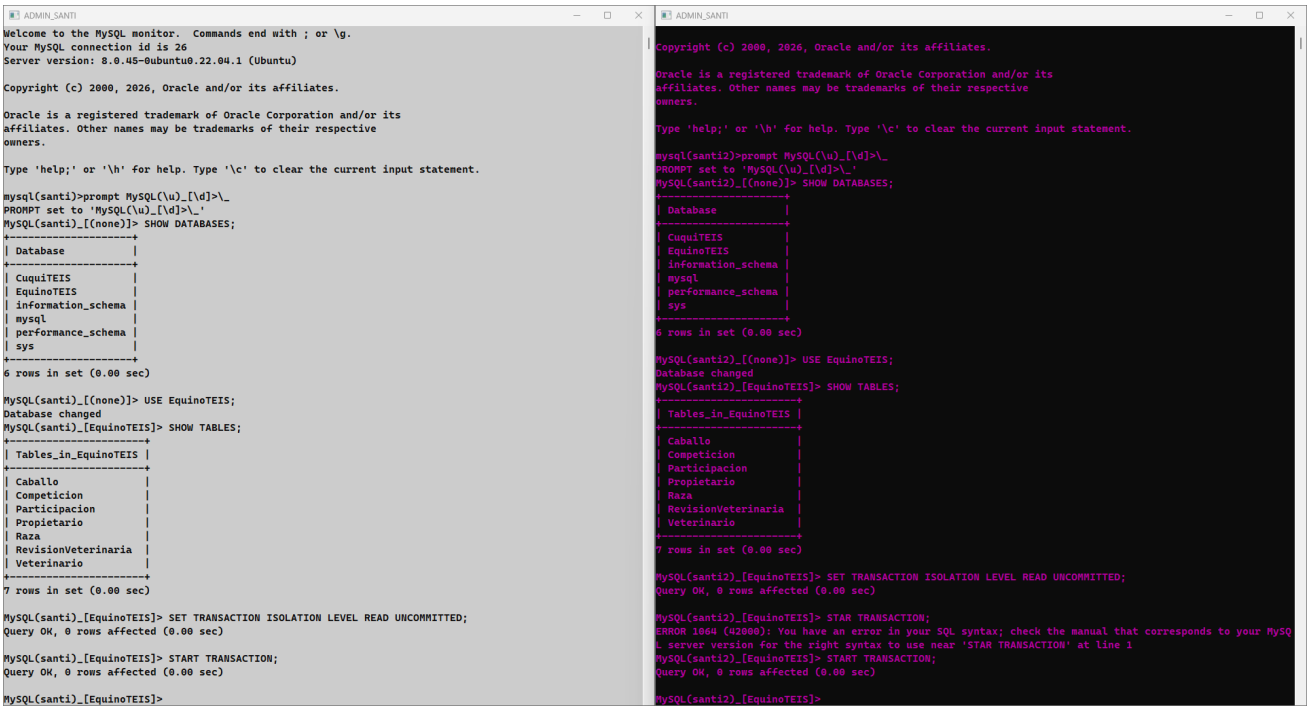
  berdeal_outes_fulgencia_BD05_Tarea

```
---------------------------------------------------------------------------------------- 0.
Conexión y configuración inicial
-- ----------------------------------------------------------------------------------------
USE EquinoTEIS;

-- Niveles de aislamiento recomendados

-- MySQL:    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- ----------------------------------------------------------------------------------------
-- 1. Pasos a seguir
--
-- 1. Abrir dos sesiones simultáneas: T1 y T2

--    * En MySQL  -> emplea sesiones MySQL 8.0 Command Line Client
-- --
-- 2. Inicia en el nivel de aislamiento que desees en ambas sesiones T1 y T2, esperando el
Query Ok.
-- --
-- 3. Ejecuta ahora los comandos asociados (u otros similares) a cada fenómeno paso a paso.
Ten cuidado al ejecutarlos para segurar que sea en la sesión indicada.
-- --
-- 4. Realiza las capturas de pantalla que consideres adecuadas.
-- --
-- 5. En resultado indica si es el esperado al Fenómeno asociado, y si no lo es indica a qué
se debe y cómo se podría evitar.
-- --
-- 6. Si quieres, puedes comparar resultados en Oracle vs MySQL.

-- ----------------------------------------------------------------------------------------
-- 2. TABLA DE REFERENCIA DE FENÓMENOS (con ejemplos de T1/T2)
-- ----------------------------------------------------------------------------------------
```

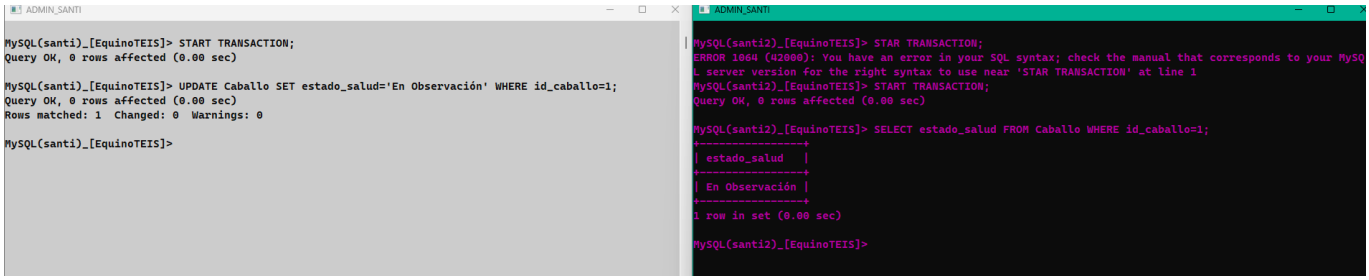
FENÓMENO	SGBD	NIVEL DE AISLAMIENTO	EJEMPLO CABALLO	OBSERVACIÓN
Dirty Read	MySQL	READ UNCOMMITTED	T1 actualiza estado_salud id=1; T2 lee antes de commit	Oracle no permite dirty read|
Dirty Write	MySQL	READ UNCOMMITTED	T1 actualiza estado_salud id=4; T2 actualiza mismo campo	Oracle no permite dirty writE
Non-repeatable Read	Oracle/ MySQL	READ COMMITTED	T1 lee peso id=2; T2 actualiza; T1 vuelve a leer	Lectura no repetible
Lost Update	Oracle/ MySQL	READ COMMITTED	T1 lee y actualiza peso id=3; T2 lee y actualiza	Último commit sobrescribe primer update
Phantom Read	Oracle/ MySQL	REPEATABLE READ	T1 consulta altura≥1.70; T2 inserta id=21; T1 vuelve a consultar	Aparece fila nueva
Write Skew / Serialization Anomaly	Oracle/ MySQL	REPEATABLE READ / SERIALIZABLE	T1 y T2 actualizan caballos a 'Bueno' simultáneamente	Ningún caballo queda con "Excelente"

BD05. ENUNCIADO TAREA pruebas de Concurrencia en los datos de equinoTEIS

```
-- 3. FENÓMENOS PASO A PASO
-------------------------------------------------------------------------------------------
-- En cada fenómeno hay un -- Resultado: en el que deberás indicar la anomalía que has
observado
```

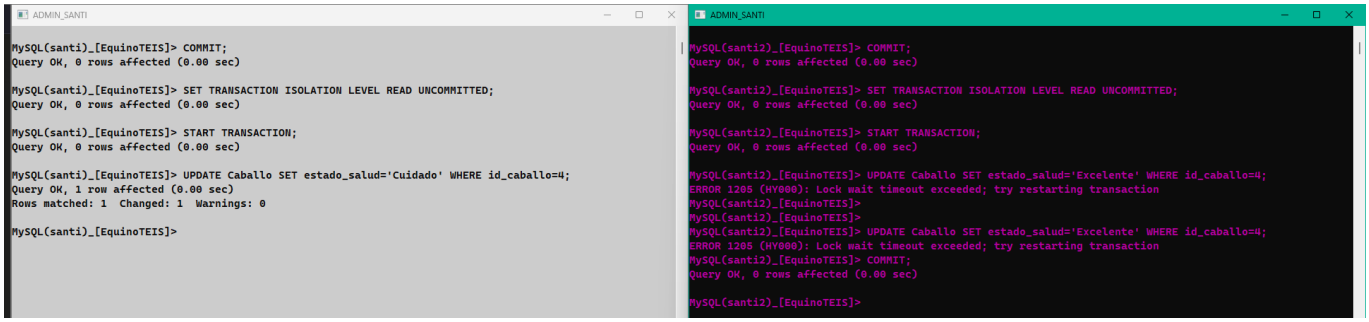



```
-- ---------------------------------------
-- FENÓMENO 1: Dirty Read
-- ---------------------------------------
-- Transacción 1 (T1)
UPDATE Caballo SET estado_salud='En Observación' WHERE id_caballo=1;

-- Transacción 2 (T2)
SELECT estado_salud FROM Caballo WHERE id_caballo=1;
```

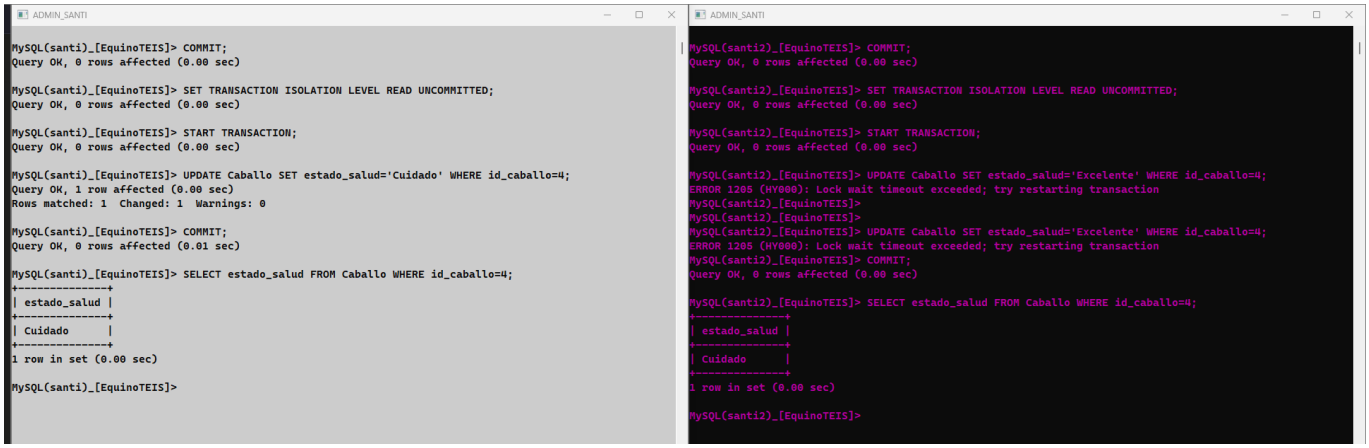



```
-- Transacción 1 (T1)
COMMIT;

-- Resultado:

-- T2 lee el valor 'En Observación' antes de que T1 haga COMMIT.
-- Es el resultado esperado de un Dirty Read bajo READ UNCOMMITTED.
-- Se podría evitar con un READ COMMITTED, por ejemplo.
```

```
-- ---------------------------------------
-- FENÓMENO 2: Dirty Write
-- ---------------------------------------
-- Transacción 1 (T1)
UPDATE Caballo SET estado_salud='Cuidado' WHERE id_caballo=4;

-- Transacción 2 (T2)
UPDATE Caballo SET estado_salud='Excelente' WHERE id_caballo=4;
COMMIT;
```




```
-- Transacción 1 (T1)
COMMIT;
```

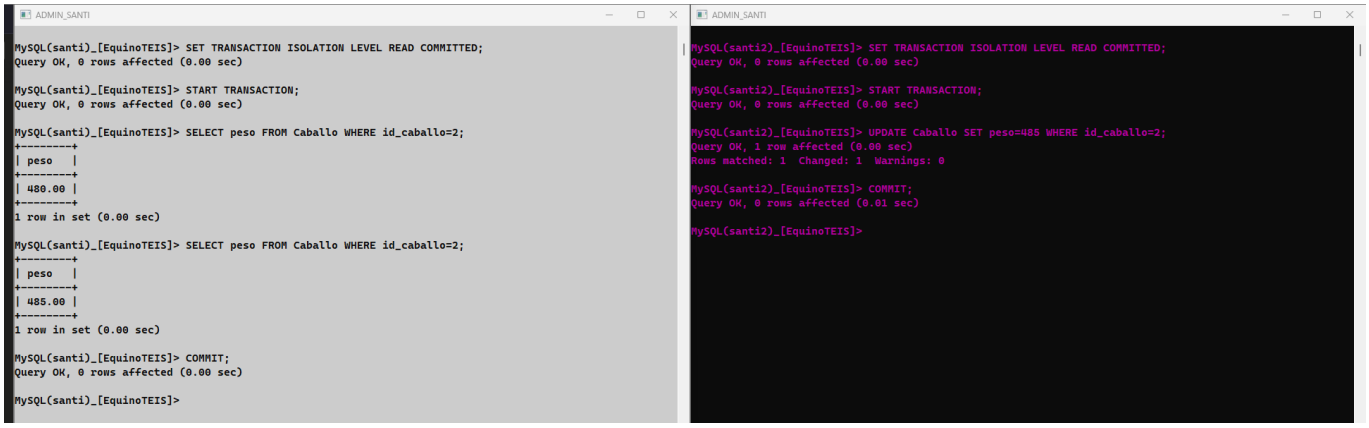


-- Resultado:

-- T2 queda bloqueada al intentar actualizar la misma fila que T1 no ha commiteado.
-- MySQL previene el Dirty Write con locks de escritura incluso en READ UNCOMMITTED.
-- La transacción en SANTI2 quedará bloqueda en el paso 2 hasta que SANTI1 haga COMMIT.
-- El valor final depende de qué transacción commitea última.

BD05. ENUNCIADO TAREA pruebas de Concurrencia en los datos de equinoTEIS

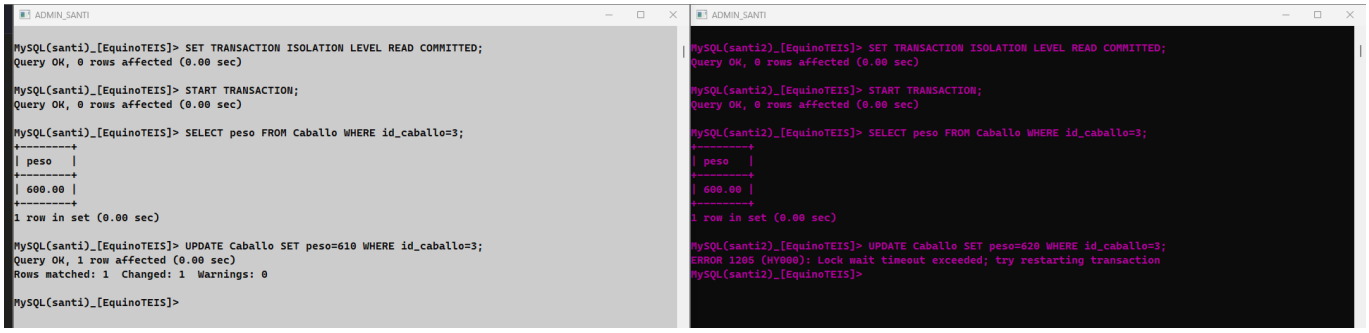
```
-- ---------------------------------------
-- FENÓMENO 3: Non-repeatable Read
-- ---------------------------------------
-- Transacción 1 (T1)
SELECT peso FROM Caballo WHERE id_caballo=2;

-- Transacción 2 (T2)
UPDATE Caballo SET peso=485 WHERE id_caballo=2;
COMMIT;

-- Transacción 1 (T1)
SELECT peso FROM Caballo WHERE id_caballo=2;
COMMIT;
```



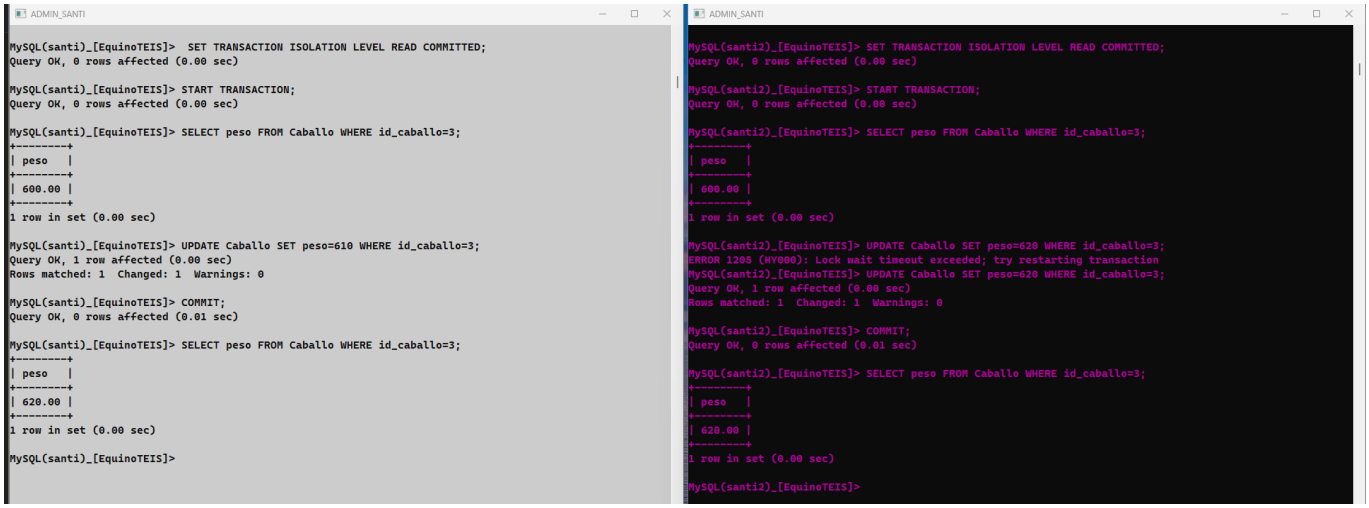
-- Resultado:

-- T1 obtiene un valor de peso distinto en su segunda lectura tras el COMMIT de T2.
-- Es el resultado esperado de un Non-repeatable Read bajo READ COMMITTED.
-- Para evitarlo: usar REPEATABLE READ o superior.

```
-- ---------------------------------------
-- FENÓMENO 4: Lost Update
-- ---------------------------------------
-- Transacción 1 (T1)
SELECT peso FROM Caballo WHERE id_caballo=3;

-- Transacción 2 (T2)
SELECT peso FROM Caballo WHERE id_caballo=3;

-- Transacción 1 (T1)
UPDATE Caballo SET peso=610 WHERE id_caballo=3;

-- Transacción 2 (T2)
UPDATE Caballo SET peso=620 WHERE id_caballo=3;
```



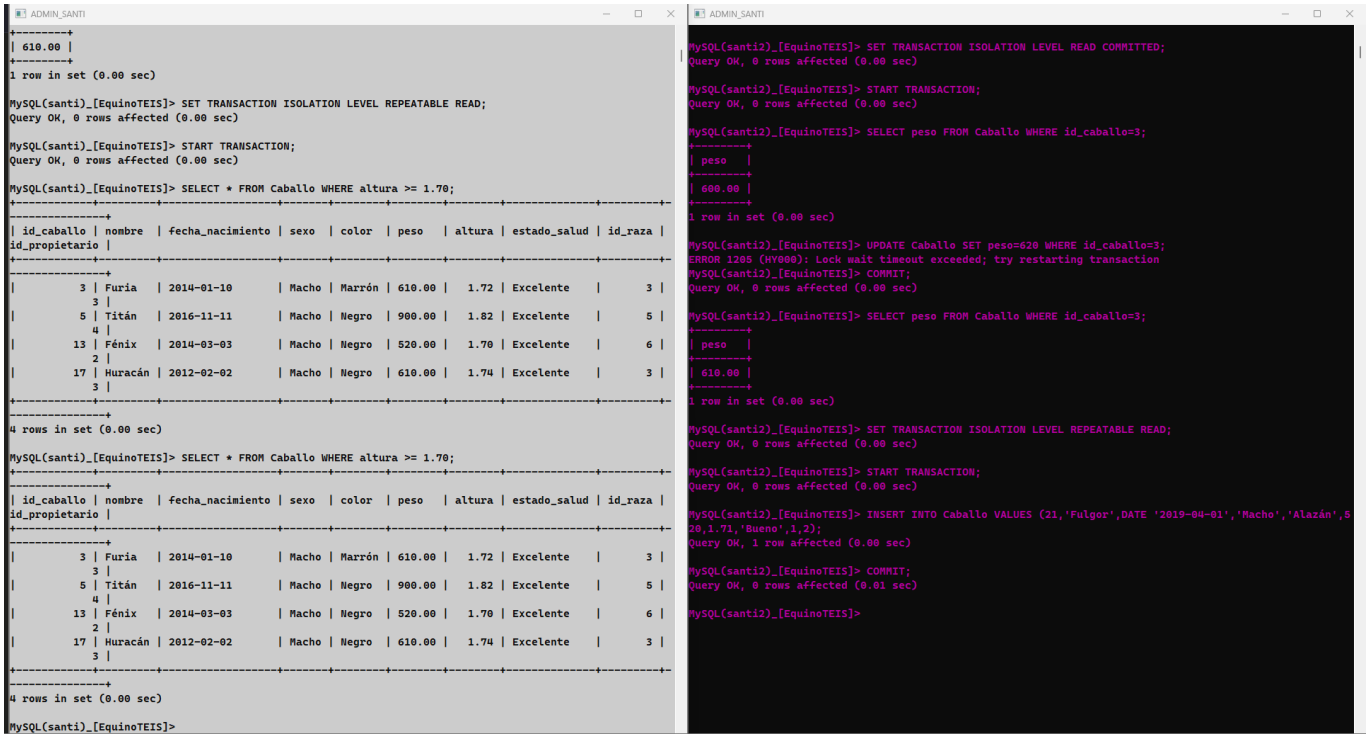

```
-- Transacción 2 (T2)
COMMIT;

-- Transacción 1 (T1)
COMMIT;
```




```
-- Resultado:
-- El UPDATE de T1 (peso=610) se pierde. El valor final es 620 (T2).
-- T2 estuvo bloqueada hasta el COMMIT de T1, pero al desbloquearse sobreescribió el valor.
-- Es el resultado esperado de un Lost Update bajo READ COMMITTED.
-- Para evitarlo: usar SELECT ... FOR UPDATE o SERIALIZABLE.
```

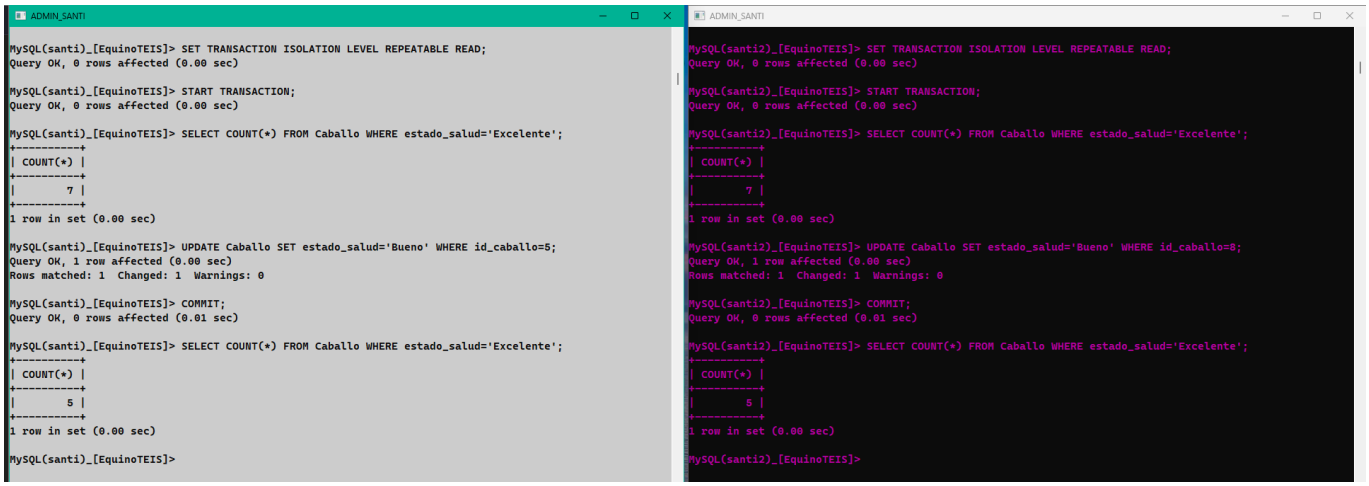
BD05. ENUNCIADO TAREA pruebas de Concurrencia en los datos de equinoTEIS

```
-- ---------------------------------------
-- FENÓMENO 5: Phantom Read
-- ---------------------------------------
-- Transacción 1 (T1)
SELECT * FROM Caballo WHERE altura >= 1.70;

-- Transacción 2 (T2)
INSERT INTO Caballo VALUES (21,'Fulgor',DATE '2019-04-
01','Macho','Alazán',520,1.71,'Bueno',1,2);
COMMIT;

-- Transacción 1 (T1)
SELECT * FROM Caballo WHERE altura >= 1.70;
```

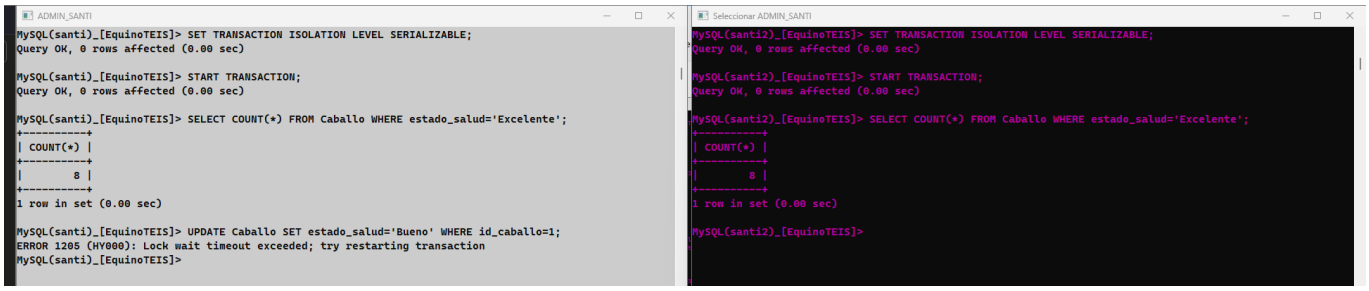



```
-- Resultado:

-- T1 obtiene el mismo conjunto de filas en ambos SELECT, sin ver la fila insertada por T2.
-- MySQL con REPEATABLE READ previene el Phantom Read en lecturas normales.
-- Para forzar el fenómeno: usar SELECT ... FOR UPDATE en T1.
-- Con SERIALIZABLE, T2 quedaría bloqueada directamente.
```

```
-- ---------------------------------------
-- FENÓMENO 6: Write Skew / Serialization Anomaly
-- ---------------------------------------
-- Restricción lógica: al menos un caballo debe estar con estado_salud='Excelente'

-- Transacción 1 (T1)
SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=1;

-- Transacción 2 (T2)
SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=3;

-- Transacción 1 y 2
COMMIT;
```




```
-- Si se utiliza SERIALIZABLE.
```




```
-- FENÓMENO 6 - Resultado:
-- T1 y T2 leen COUNT=7 caballos en 'Excelente'. Cada una decide cambiar uno distinto a
'Bueno'.
-- Al commitear ambas, el COUNT final es 5 en lugar del 6 que cada transacción esperaba.
-- Ninguna violó la regla individualmente, pero juntas sí: es el Write Skew.
-- Para evitarlo: usar SERIALIZABLE (una transacción quedará bloqueada hasta que la otra
commitee, el SELECT COUNT en T2 ya es suficiente para BLOQUEAR el UPDATE en T1).
```